

FULL LEGAL NAME	LOCATION (COUNTRY)	EMAIL ADDRESS	MARK X FOR ANY NON-CONTRIBUTING MEMBER
Nites Kumar	India	Niteshkr2209@gmail.com	
Eugene Ablordeppey	Ghana	eugeneablord@gmail.com	
Rushang Kasundra	India	180130107026_2018@gecg28.ac.in	

Statement of integrity: By typing the names of all group members in the text boxes below, you confirm that the assignment submitted is original work produced by the group (excluding any non-contributing members identified with an “X” above).

Team member 1

Nitesh Kumar

Team member 2

Eugene Ablordeppey

Team member 3

Rushang Kasundra

Use the box below to explain any attempts to reach out to a non-contributing member. Type (N/A) if all members contributed.

Note: You may be required to provide proof of your outreach to non-contributing members upon request.

N/A

FULL LEGAL NAME.....	0
Issue 1: Optimizing Hyperparameters.....	2
Technical Section.....	2
Introduction.....	2
Hyperparameter Optimization/Tuning.....	2
Hyperparameter Optimization/Tuning Techniques.....	3
Hyperparameters Used in Project 1.....	3
Illustration of Hyperparameter Optimization.....	5
Non-Technical Section.....	9
Importance of Hyperparameter Optimization.....	9
Applications of Hyperparameter Optimization in Finance.....	9
Issue 2: Optimizing the Bias Variance Tradeoff.....	11
Technical Section.....	11
2.1 What Bias and Variance Actually Measure.....	11
2.2 A Common Diagnostic Approach Across All Three Models.....	11
2.3 Ridge Regression: Regularization Strength (α).....	12
2.4 Classification Tree: Max Depth and Pruning Strength.....	14
2.5 LDA vs Neural Network: Model Capacity and Generalization.....	16
2.6 Summary: Bias –Variance Tradeoff Across All Three Models.....	17
Non-Technical Section.....	18
How Ridge Regression Handled the Tradeoff.....	18
How the Classification Tree Handled the Tradeoff.....	19
How the Simple Model (LDA) Compared to the Flexible Model (Neural Network).....	19
What this means for the Strategist’s Question.....	19
References.....	20
Issue 3: Applying Ensemble Learning - Bagging, Boosting, and Stacking.....	21
Technical Section.....	21
3.1 Purpose of ensemble learning.....	21
3.2 Main ensemble approaches.....	21
3.3 Models and hyperparameters used.....	22
3.4 Quantitative results.....	23
Non-Technical Section.....	24
References.....	24

Issue 1: Optimizing Hyperparameters

Technical Section

Introduction

Hyperparameter optimization is a key part of building robust machine learning models. Unlike the model parameters, which are learned from the dataset during training, hyperparameters are defined before training starts and affect the way the learning algorithm learns from the data. Appropriate values for hyperparameters can enhance the performance of the model, prevent overfitting, and improve the generalization capabilities of the model. In finance, where the environment is constantly changing and noisy, the proper selection of hyperparameters is crucial in building a good predictive model.

The purpose of hyperparameter optimization is to find the right values for the hyperparameters that result in optimal performance on new data as opposed to optimizing only the training accuracy.

Hyperparameter Optimization/Tuning

Hyperparameter tuning can be understood as the systematic process of identifying the optimal setting of hyperparameters prior to training the machine learning model. The hyperparameters regulate the complexity of a learning algorithm and control its behavior; unlike the parameters which are learned automatically through model training, hyperparameters need to be defined by the practitioner. Examples include the regularization parameter (α) in Ridge Regression, the max depth of the tree in the Decision Tree, the penalty parameter (C) in Support Vector Machines, and the learning rate in the Neural Network. Tuning allows the model to have the highest predictive power while reducing both underfitting and overfitting. Before tuning, a hyperparameter search space is defined, indicating the candidate values or distribution from which the search algorithm would identify the optimal hyperparameters.

The four main steps of tuning are:

1. Define the hyperparameter search space.
2. Assess the candidate set of hyperparameters via cross-validation.
3. Compare model performances using a validation metric.
4. Choose the optimal configuration and retrain the model on the full training dataset.

Hyperparameter Optimization/Tuning Techniques

There have been various approaches devised that would automatically select optimal hyperparameters. The difference between the methods lies in the process of examining different combinations of hyperparameters and the computing power necessary to do so.

- The method of manual search uses trial and error. In other words, the user changes the value of hyperparameters, tests the model by cross-validation, and compares the performance. This method is easy to implement and allows understanding how each particular hyperparameter affects the model; however, it cannot be used for complex models due to its exponential complexity.
- Grid Search considers all possible combinations of hyperparameter values in a given search space. As each combination is evaluated, Grid Search is usually very accurate. Nonetheless, this thorough search process is computationally intensive, especially if there are several hyperparameters or a huge search space.
- Random Search randomly samples hyperparameter combinations from predefined distributions instead of evaluating every possible combination. By exploring only a subset of the search space, Random Search significantly reduces computation time while often achieving performance comparable to Grid Search, especially when only a few hyperparameters strongly influence model accuracy.
- Bayesian Optimization is an improvement of the search process through learning from prior evaluations. It learns the best combination of parameters that have better chances of providing good results rather than randomly or exhaustively checking the whole search space like grid search and random search. Hence, Bayesian Optimization normally takes fewer evaluations to converge.

Hyperparameters Used in Project 1

The machine learning models investigated in Project 1 each relied on different hyperparameters that influence model performance. Table 1 summarizes the primary hyperparameters associated with each model

Model	Key Hyperparameters
Ridge Regression	Regularization parameter (α)
K-Means Clustering	Number of clusters (K), initialization method
Principal Component Analysis	Number of principal components
Classification Tree	Maximum tree depth, minimum samples per leaf
Linear Discriminant Analysis	Solver, shrinkage parameter
Support Vector Machine	Penalty parameter (C), kernel type, gamma (γ)
Neural Network	Learning rate, hidden layer size, regularization parameter, number of epochs

Mathematical Formulation of Hyperparameter Optimization

Hyperparameter tuning can be treated as an optimization problem where the goal is to find the configuration of the hyperparameters resulting in the highest predictive power on unseen data. Instead of optimizing the parameters of the model, the optimization technique scans a certain hyperparameter search space and assesses the performance of each configuration by using a validation set or cross-validation. The best hyperparameters are those yielding the lowest value of validation loss function or the maximum value of the specified performance metric.

The general optimization problem is expressed as:

$$\theta^* = \arg \min_{\theta \in \Theta} L(f(X_{\text{train}}; \theta), Y_{\text{val}})$$

where:

- θ represents the set of hyperparameters to be optimized.
- Θ denotes the predefined hyperparameter search space.
- $f(X_{\text{train}}; \theta)$ represents the machine learning model trained using the hyperparameter configuration θ .
- $L(\cdot)$ is the validation loss function, such as Mean Squared Error (MSE) for regression problems or Cross-Entropy Loss for classification problems.
- Y_{val} represents the observed output values in the validation dataset.

This equation means that the objective of hyperparameter optimization is to determine the hyperparameter setting that will minimize the validation loss of the model, thus enabling it to be more generalizable to unseen data.

Since hyperparameter optimization is commonly performed using K-fold cross-validation, the optimization objective can also be expressed as maximizing the average validation score across all folds:

$$\theta^* = \arg \max_{\theta \in \Theta} \frac{1}{K} \sum_{i=1}^K \text{Score}_i(\theta)$$

where:

- K is the number of cross-validation folds.
- $\text{Score}_i(\theta)$ is the performance metric (e.g., accuracy, F1-score, ROC-AUC, or R^2) obtained on the i^{th} validation fold using hyperparameter configuration θ .

Through cross-validation, performance can be obtained through the average performance for several validation folds. This method ensures that there is no bias toward a certain parameter setting that performs extremely well only on a particular part of the training data. The chosen set of parameters will therefore generalize well to new financial data.

Illustration of Hyperparameter Optimization

To illustrate the application of hyperparameter optimization, an SVM classifier was developed to predict the next-day direction of JPMorgan Chase & Co. (JPM) stock returns. The objective of the model is to classify whether the following day's return will be positive (1) or non-positive (0) using historical market information. The dataset used for this analysis was jpm_us_d.csv, which contains daily stock market observations, including Date, Open, High, Low, Close, and Volume. The dataset contains 14,159 observations. Since raw stock prices alone may not fully capture market behavior, additional technical indicators were created to improve the predictive capability of the model.

The engineered features used in the SVM model were:

Table 1. Description of Model Input Features

Feature	Description
Previous Day Return	Return percentage from the previous trading day
Volume	Number of shares traded
Volatility	20-day rolling standard deviation of returns

MA_20	20-day moving average of closing price
RSI	Relative Strength Index, a momentum indicator
Daily Return	Daily percentage return calculated from JPM closing prices

The target variable was defined as:

- 1: Next-day return is positive
- 0: Next-day return is zero or negative

The dataset was split chronologically into training and testing sets, with 80% of observations used for training (11,311 samples) and 20% reserved for testing (2,828 samples).

Hyperparameter Optimization

The SVM classifier performance depends mainly on two hyperparameters:

- C (Regularization Parameter): Controls the trade-off between maximizing the decision boundary margin and minimizing classification errors. Smaller values create simpler models, while larger values allow the model to fit more complex patterns.
- Gamma (γ): Controls how much influence each training observation has on the decision boundary. Smaller values create smoother boundaries, while larger values allow more localized and complex decision boundaries.

To identify the optimal combination of these parameters, Grid Search with five-fold cross-validation was applied. The search evaluated the following parameter values:

- C: 0.1, 1, 10, 100, 1000
- Gamma (γ): 0.0001, 0.001, 0.01, 0.1, 1

A total of 25 hyperparameter combinations were evaluated, with each combination tested using five-fold cross-validation.

Grid Search Results

The Grid Search process produced validation accuracy scores for each combination of C and γ . The combination with the highest cross-validation accuracy was selected as the optimal SVM configuration.

Table 2. Grid Search Results for SVM Hyperparameter Optimization

Rank	C	Gamma (γ)	Validation Accuracy	Validation Accuracy (%)
1	0.1	0.0001	0.540801	54.08%
2	0.1	0.001	0.540801	54.08%
3	1	0.0001	0.540094	54.01%
4	100	0.001	0.528511	52.85%
5	1	0.01	0.528423	52.84%

6	0.1	0.01	0.526477	52.65%
7	10	0.001	0.525770	52.58%
8	100	0.0001	0.525063	52.51%
9	1000	0.0001	0.524886	52.49%
10	1	0.001	0.524709	52.47%

Hyperparameter Performance Heatmap

To visualize the impact of different hyperparameter combinations, a heatmap was generated using the Grid Search results. Darker regions represent higher validation accuracy.

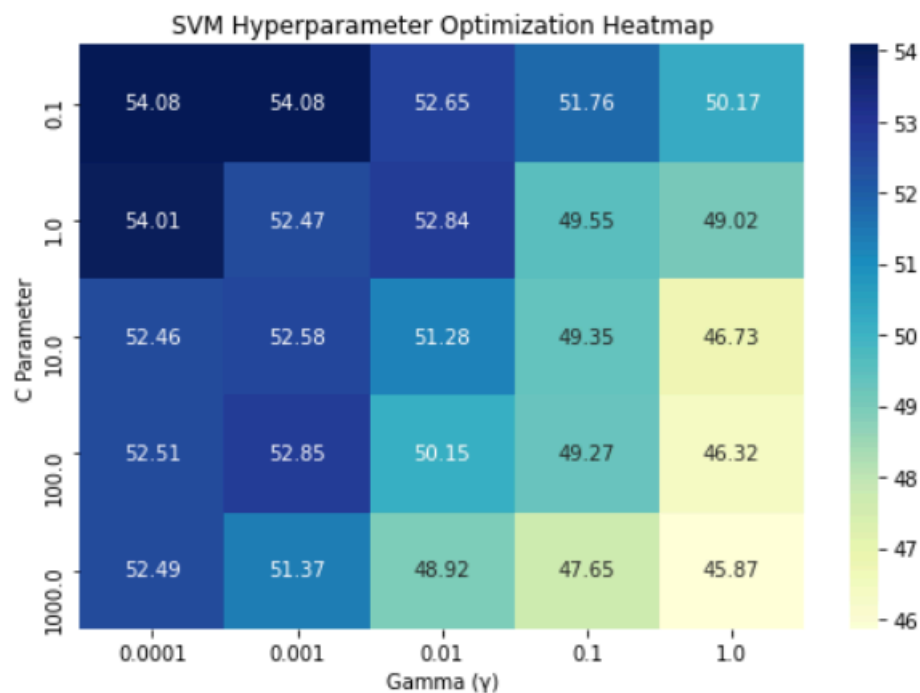


Figure 1. Heatmap of SVM Grid Search Cross-Validation Accuracy

The heatmap provides a clear view of how different combinations of C and γ affect model performance. The optimal region represents the parameter combination that provides the best balance between model flexibility and predictive accuracy.

Effect of C on Model Performance

A validation curve was used to examine how changing the C parameter influences model performance while keeping gamma fixed at the optimized value.

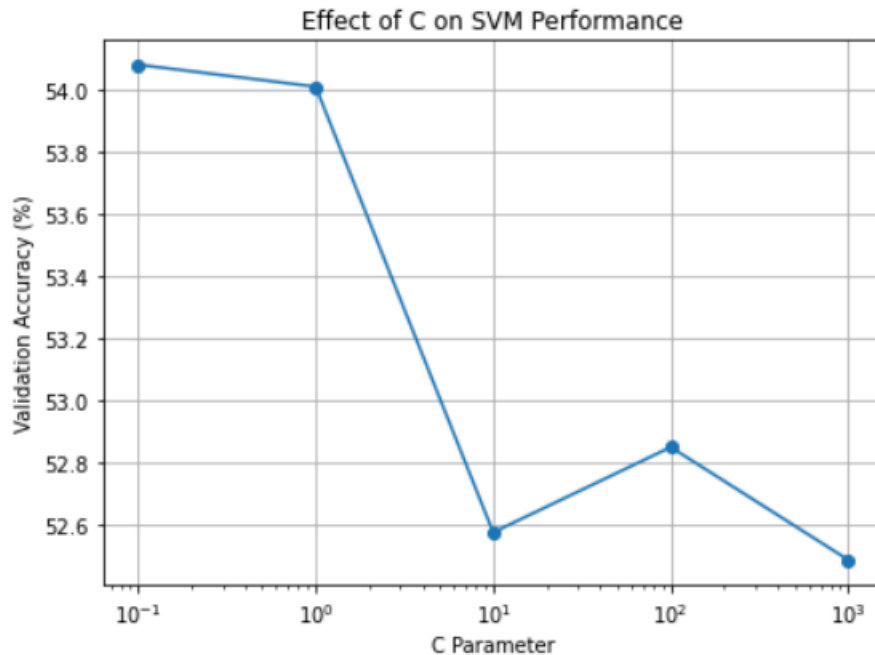


Figure 2. Validation Curve Showing the Effect of C on SVM Accuracy

The results show the relationship between model complexity and predictive performance.

Decreasing C initially improves accuracy because the model can better capture patterns in the data.

Effect of Gamma on Model Performance

A second validation curve was created to evaluate the effect of gamma on SVM performance.

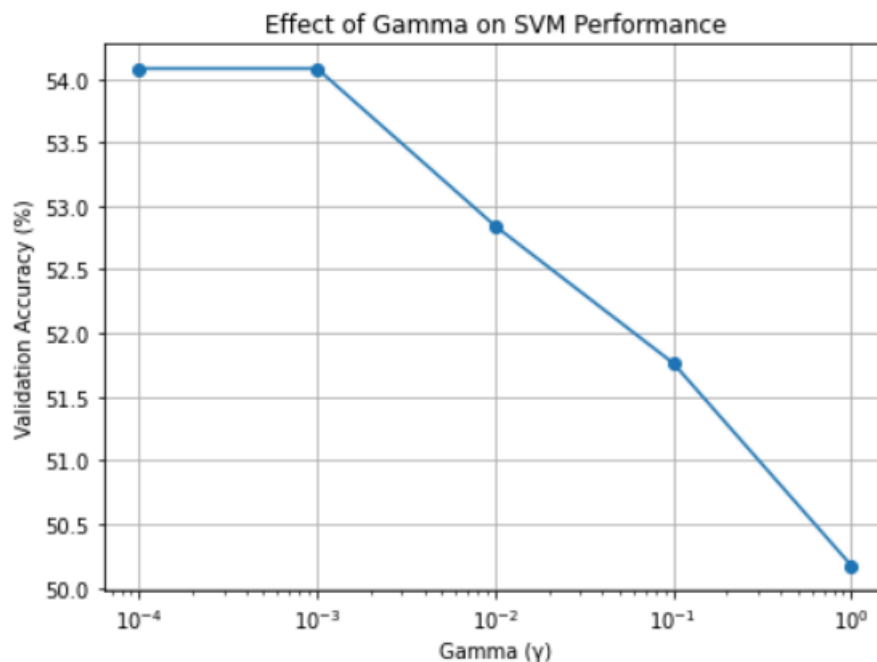


Figure 3. Validation Curve Showing the Effect of Gamma on SVM Accuracy

The results demonstrate the importance of selecting an appropriate gamma value. Very small values may produce an overly simple decision boundary, while very large values may cause the model to become too sensitive to individual observations and increase the risk of overfitting.

Impact of Hyperparameter Optimization

The example shows how hyperparameter optimization is useful for finance-related applications of machine learning. While selecting model parameters manually was not an efficient way of doing things, the Grid Search technique evaluates various combinations of C and γ and finds the most effective combination for validation results. This way, the SVM model becomes more flexible but still remains generalizable. The choice of proper parameters makes sure that both underfitting and overfitting are avoided.

Non-Technical Section

In Machine Learning, hyperparameter optimization is an important process that improves a model's ability to make reliable predictions on new data. Similar to calibrating a financial forecasting tool before using it for investment decisions, optimizing model settings helps ensure that predictions are based on meaningful patterns rather than noise. In finance, where inaccurate predictions can lead to poor investment decisions, having a well-configured model is essential.

Importance of Hyperparameter Optimization

Hyperparameter optimization improves the overall reliability and effectiveness of machine learning models. By selecting appropriate model settings, the model can identify useful patterns in financial data, improve prediction accuracy, and reduce the likelihood of making unstable decisions. This process also helps prevent overfitting, where a model performs well on historical data but fails when applied to new market conditions. For portfolio strategists, optimized models provide greater confidence that machine learning outputs can support investment decisions, risk assessment, and forecasting activities.

Applications of Hyperparameter Optimization in Finance

Prediction quality in financial applications has direct consequences for business outcomes, making hyperparameter optimization particularly consequential in this domain.

- Management of portfolios: Machine learning models improved for portfolio management improve the identification of investment opportunities through more reliable predictions of asset behavior and market trends.
- Risk Management: Financial institutions can use improved models to better estimate risks, including credit default probabilities, market exposure, and potential losses.
- Algorithmic Trading: Algorithmic trading employs predictive models to detect market patterns and execute trades accordingly. Proper optimization improves trading signals by mitigating the influence of random market fluctuations.
- Fraud Detection: Improved models improve the detection of unusual transaction patterns while reducing false alerts, thereby enabling financial institutions to protect against fraudulent activities.

Challenges and Considerations

Although hyperparameter optimization provides significant benefits, it also presents several challenges. Searching through different model configurations can require substantial computational resources and time, especially for complex models with many possible settings.

In addition, financial markets are constantly changing, meaning a model that performs well using historical data may not always perform equally well in the future. Therefore, optimized machine learning models should be regularly evaluated and combined with financial expertise to support effective decision-making.

Journals

1. Yu, Tong, and Hong Zhu. "Hyper-Parameter Optimization: A Review of Algorithms and Applications." *arXiv*, 2020, <https://arxiv.org/abs/2003.05689>.
2. Bergstra, J., & Bengio, Y. (2012). *Random search for hyper-parameter optimization*. *Journal of Machine Learning Research*, 13(10), 281–305. <http://jmlr.org/papers/v13/bergstra12a.html>
3. Feurer, M., Hutter, F. (2019). Hyperparameter Optimization. In: Hutter, F., Kotthoff, L., Vanschoren, J. (eds) *Automated Machine Learning*. The Springer Series on Challenges in Machine Learning. Springer, Cham. https://doi.org/10.1007/978-3-030-05318-5_1
4. Bischl, B., Binder, M., Lang, M., Pielok, T., Richter, J., Coors, S., Thomas, J., Ullmann, T., Becker, M., Boulesteix, A.-L., Deng, D., & Lindauer, M. (2021). Hyperparameter Optimization: Foundations, Algorithms, Best Practices and Open Challenges. *Arxiv.Org*. <https://doi.org/10.48550/arXiv.2107.05847>

Issue 2: Optimizing the Bias Variance Tradeoff

The portfolio strategists posed a key question regarding the models shown in GWP1: what performance can we expect from the models when faced with unseen data? The question poses the bias-variance tradeoff-- the tradeoff between a model that is overly simplistic and cannot capture the nuance (high bias) vs a model that is too tailored to a specific training sample (high variance). This section attempts to analyze this tradeoff for three of the seven models in GWP1: Ridge Regression, the Classification Tree, and the LDA versus Neural Networks comparison. This comparison should be adequate to capture regression, a tree-based classifier, and a tradeoff between a simple vs flexible model.

Technical Section

2.1 What Bias and Variance Actually Measure

A model's expected error on a fresh data point splits cleanly into three pieces:

$$\text{Expected Test Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$$

- **Bias** is the average deviation of a model's prediction from the actual answer. It is possible for a model to be retrained many times on the same type of data, but it would still carry the same error. Bias exists in the model as a straight line to describe a curved relationship, for example, is always wrong no matter how much data is put behind it.
- **Variance** describes how much predictions change when a model is trained on data samples of a different subset. A model that has memorized the training data set will create a significantly different model if a different training data set has been given to it.
- **Irreducible noise** describes the randomness in a data set that no model can overcome.

In a data set, training data is a good approximation of bias (a model with high bias will not fit the training data), and the difference between training data and test data is a good representation of variance (a model with high variance will closely fit the training data, but will not fit the test data).

2.2 A Common Diagnostic Approach Across All Three Models

The same three-part diagnostic was applied to each model:

- Sweep the complexity control while monitoring training and test performance. Where the two diverge indicates an over/underfitting problem. A complexity control in Ridge loss refers to the ridge loss added to the regression. A complexity control in the tree refers to the max depth and pruning strength. For the neural networks, it refers to the hidden-layer width.
- Run a Grid/Random Search with scikit-learn's Ridge model. However, instead of using a normal k-fold cross-validation, use scikit-learn's TimeSeriesSplit. With financial time series, there are real dangers in using a completely random k-fold. It may make the model look really good when in fact it isn't.
- Use a learning curve to diagnose a persistent training and test performance gap. In the learning curve you would plot training and test performance against the number of training examples. A gap that would likely close with more data indicates a variance problem. A gap that persists with more data is most likely a bias or feature problem.

2.3 Ridge Regression: Regularization Strength (alpha)

GWP1 demonstrated that Ridge Regression could be reliably fit on a monthly style, small, macro panel (composed of interest rates, CPI, unemployment, VIX predicting an index return) that had the same multicollinearity structure with respect to the original FRED data.

When alpha was swept from 0.01 up to 1,600 (log scale), the classic U-shaped test-error curve was realized:



Figure 2.1: Ridge Regression train vs. test MSE across the regularization path. Train error rises steadily with alpha (increasing bias); test error dips to a minimum around alpha ≈ 11.7 before rising again (variance reduction, then over-shrinkage).

- With the smallest alpha tested (unregularized OLS), the Regression with 63 training rows and highly correlated predictors had a training MSE = 0.00243 with a testing MSE = 0.00314. The model was showing clear overfitting and capturing sample specific noise.
- With the best alpha value (11.66) used, training MSE = 0.00245 and testing MSE = 0.00312. A small deterioration in training error produced a significant improvement in testing error.
- With the largest alpha value ($\approx 1,600$) used, training MSE = 0.00347 and testing MSE = 0.00405. Both MSEs increased and the model became too constrained to capture the weak true signal.
- A 5-fold, time-respecting grid search selected alpha = 15.85 and was in close alignment of the single-split result, increasing confidence that the selected alpha value did not originate from a specific train/test split.

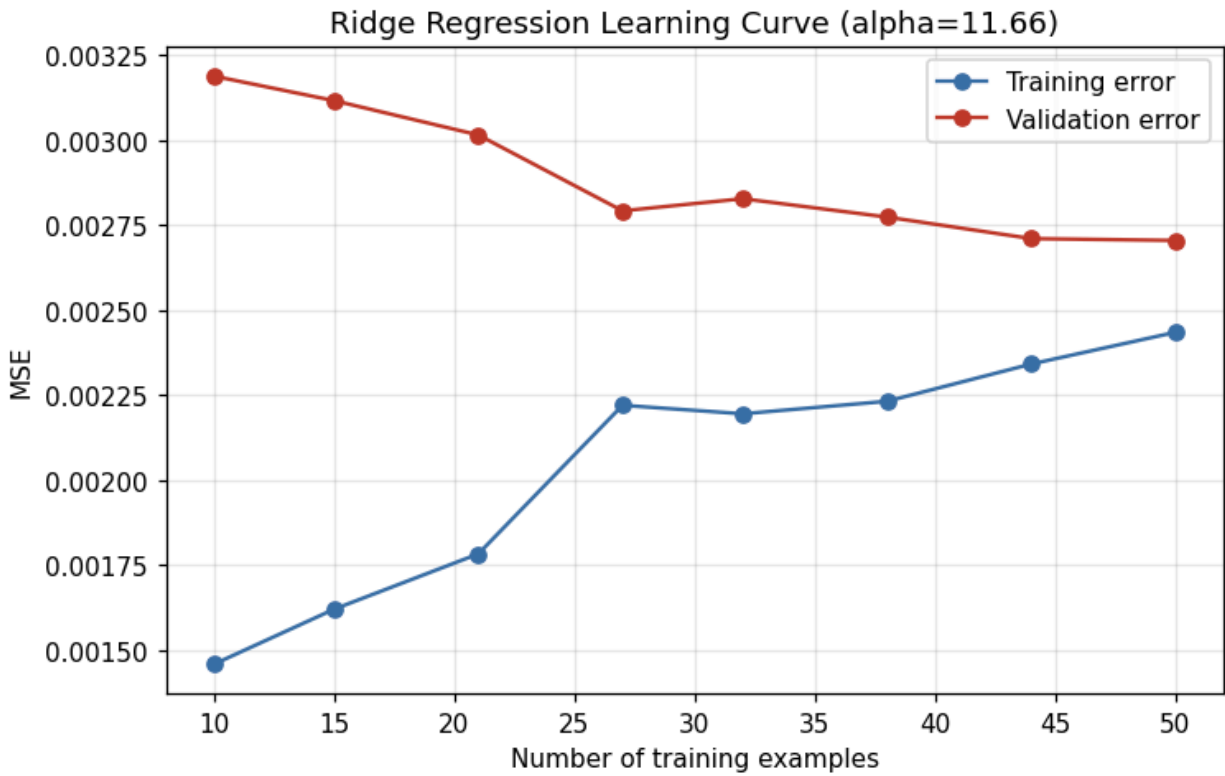


Figure 2.2: Learning curve at the tuned alpha. Training error rises slightly and validation error falls as more data is used, and the two curves move toward each other -- consistent with a model whose remaining error is partly a variance issue that more data would continue to help with.

2.4 Classification Tree: Max Depth and Pruning Strength

GWP1 Category 4 provided a 5-asset simulated panel with 4,950 pooled rows and a next-day up/down target as well as an 80/20 chronological split. This provided the basis to investigate the tree's complexity controls with greater resolution.

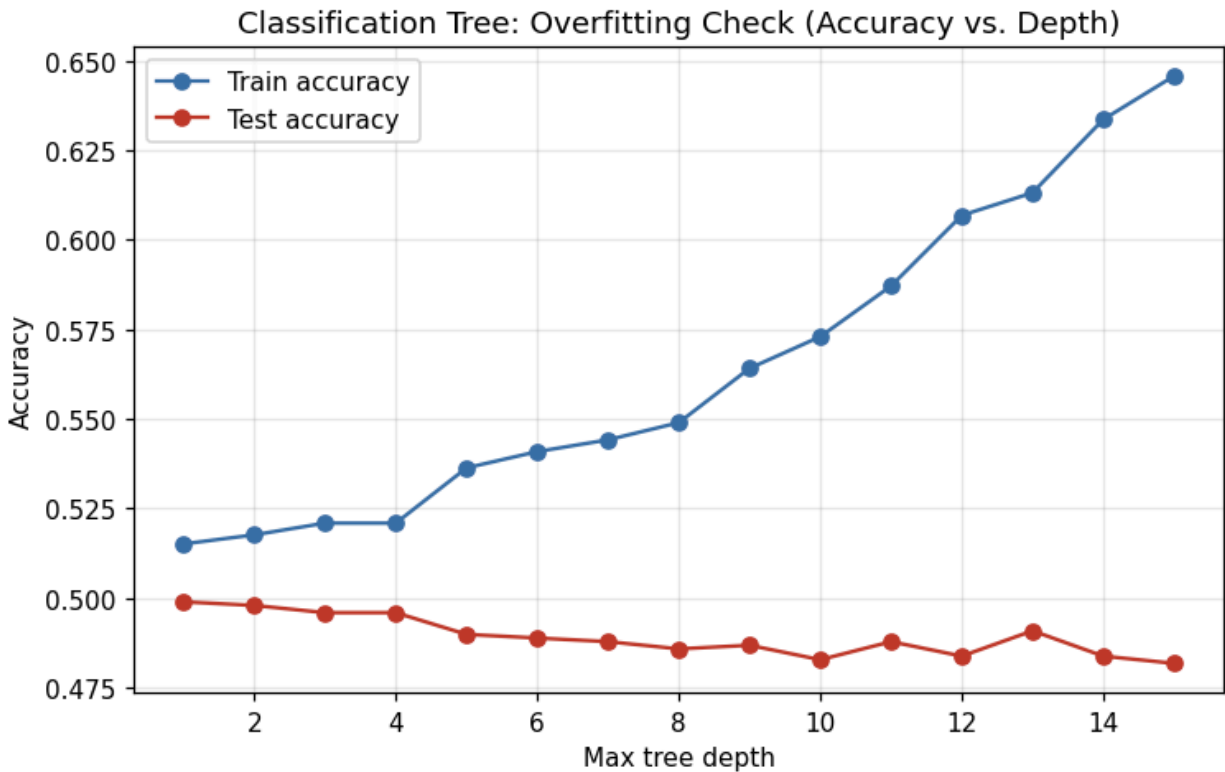


Figure 2.3: Accuracy vs. max tree depth. Training accuracy climbs from 51.5% at depth 1 to 64.6% at depth 15; test accuracy stays essentially flat around 48-50% throughout and drifts slightly downward past depth 8 -- the widening gap is variance, not improving predictive power.

- With Cost Complexity Pruning set to a single, finer control along the margin of tree depth, the best single-split result was `ccp_alpha = 0.00125`.
- A combined grid search over depth (2-8) and `ccp_alpha` (0-0.002), with ROC-AUC and 5-foldTimeSeriesSplit, chose `max_depth = 3`, `ccp_alpha = 0.002`. This achieved 49.9% accuracy and 0.500 ROC-AUC on the held-out test set, which is statistically a coin flip. This finding also aligned with GWP1, who stated that the next-day price direction is almost impossible to predict.

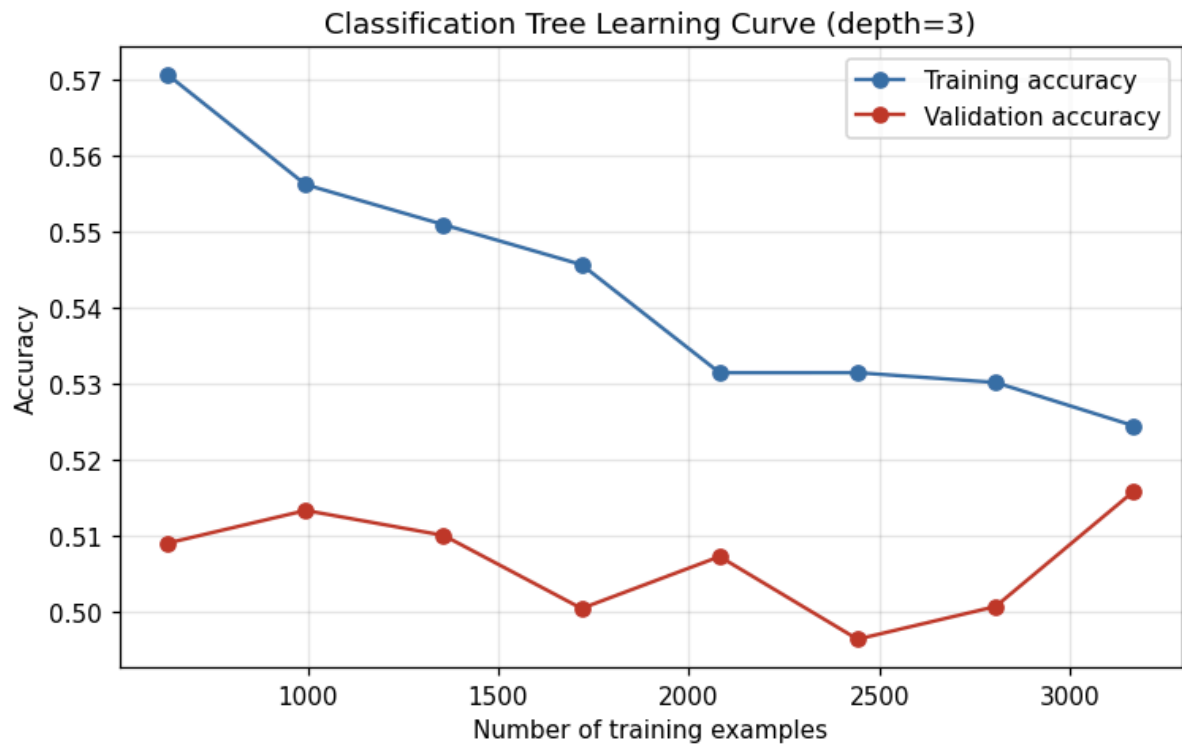


Figure 2.5: Learning curve at the tuned depth. Training and validation accuracy are already close together at the smallest sample size and stay close as more data is added -- a signature of a bias-limited (feature-limited) problem rather than one that more data alone would fix.

2.5 LDA vs Neural Network: Model Capacity and Generalization

LDA (Category 5) and Multi-Layer Perceptron (Category 7) were applied on the same SPY-style feature set to illustrate the capacity/variance relation. The validation curve described the effect on the training AUC and the cross-validated AUC as the MLP hidden-layer width was varied between 2 and 64.:

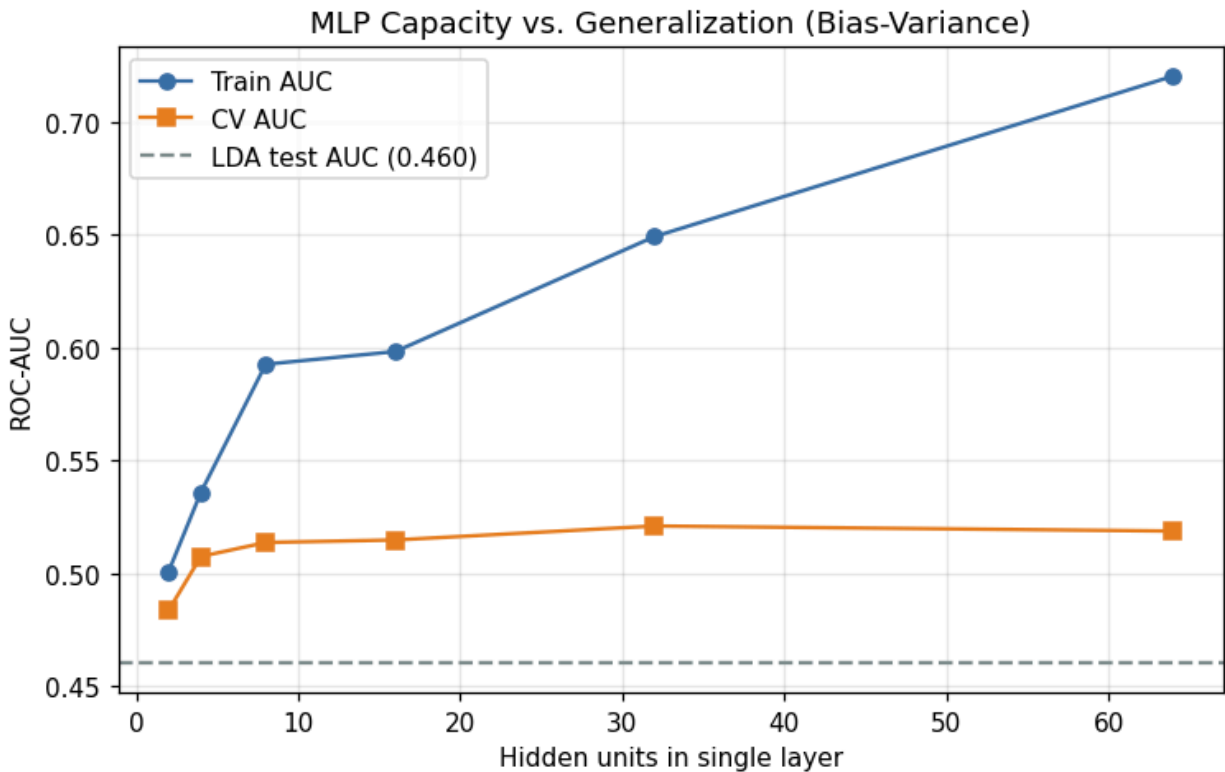


Figure 2.6: MLP capacity vs. generalization. Training AUC climbs steadily from 0.50 (2 units) to 0.72 (64 units) as capacity grows, while cross-validated AUC stays essentially flat between 0.48 and 0.52 across the entire range -- the widening gap between the two lines is variance growing with capacity, without any corresponding gain in genuine predictive power. LDA's test AUC (0.460, dashed line) sits within that same flat band.

- The grid search selected a width of 32 units by (marginal) cross-validated AUC; on the held-out test set the tuned **MLP scored 49.8% accuracy and 0.494 ROC-AUC, against LDA's 46.1% accuracy and 0.460 ROC-AUC** -- close enough that neither model is meaningfully outperforming the other, or a coin flip..
- This is the clearest three-way illustration in the report of how added flexibility (more hidden units) increases variance risk well before it produces any real accuracy gain, when the underlying signal is this weak.

2.6 Summary: Bias –Variance Tradeoff Across All Three Models

Model	Complexity control	Tuned value	Train metric	Test metric	Diagnosis

Ridge Regression	alpha (L2 penalty)	11.66	MSE = 0.00245	MSE = 0.00312	Balanced: moderate alpha cuts test error vs. near-unregularized OLS without over-shrinking
Classification Tree	max_depth / ccp_alpha	depth = 3	Acc = 0.515	Acc = 0.499	Balanced: shallow, pruned tree avoids the near-100%/50% split seen when unconstrained
MLP (Neural Net)	hidden-layer width	32 units	AUC = 0.649	AUC = 0.494	Variance grows steadily with width while CV AUC stays flat near 0.48-0.52 -- capacity isn't buying real predictive power here

All computations, code, and full console output for this section are provided in the accompanying notebook, Issue2_BiasVarianceTradeoff.ipynb, submitted separately per the assignment's code/no-code separation requirement.

Non-Technical Section

The strategists posed a simple question - how effective are these models at analyzing new data? The answer from all three models was clear. These models walked the fine line between different errors, and how each model made errors was highly valuable..

How Ridge Regression Handled the Tradeoff

- Ridge Regression, when left mostly unregularized, closely fit the training data of small monthly samples. Unfortunately, this fit did not generalize to new months of data. This case is an example of the model interpreting small and noisy samples of data as highly meaningful.
- Balanced regularization of these models alters the previous case of closely fit training data for measured general improvement in prediction. If consistent beyond a certain point, the regularization will make the model too constrained, not detecting even the weak valuable relationships in the data.
- The important consideration from this is that there are optimal levels of regularization, identifiable by performance measure on new data samples, as opposed to relying mostly on estimates.

How the Classification Tree Handled the Tradeoff

- An unconstrained model in the case of decision trees, memorized training data, and performed almost randomly on new data. This is an example of a model learning the noise from the training set.
- By capping the tree growth and removing useless branches, a more stable model was achieved. Although accuracy on new data hardly changed, which is acceptable, the extreme overfitting was addressed.
- An accuracy of giving the right answer based on a model of previous pricing to tell whether pricing of a stock will go up or down tomorrow is basically worthless. This model having that answer, and pruning, is how the tree was able to foretell that the most.

How the Simple Model (LDA) Compared to the Flexible Model (Neural Network)

- Added capacity for the hidden units of a neural net led to amazing improvements in the model's ability to interpret the training data, however, its ability to generalize to new data was about the same.
- A bounded, unyielding LDA model actually surpassed a far more layered and unbounded model that was a neural net in its ability to interpret new data.
- Business and regulatory clients tend to favor unyielding models, even if they are more simple than layered models, because they tend to trust models they can understand more, even if they are not as layered.

What this means for the Strategist's Question

Put together, the answer to "how well can these models be expected to work for predicting future cases?" is: reasonably well for Ridge Regression once properly regularized, and honestly not very well for the two models predicting next-day stock direction from price history alone -- both landed close to a coin flip once properly tuned. That's not a failure of the tuning process; it's the tuning process doing its job and reporting an honest result rather than a falsely optimistic one. The consistent theme across all three models is that the gap between training

performance and test performance is the single most useful number for judging whether a model is ready to be trusted on new data, and every model in this report was tuned by watching that gap rather than by training accuracy alone.

Before any of these models informs a real investment or lending decision, the recommended next step is the same one flagged in GWP1: re-run this analysis on real, current data, with a larger and richer feature set, before treating any of these results as an actual signal.

References

- Bellotti, Tony, and Jonathan Crook. "Support Vector Machines for Credit Scoring and Discovery of Significant Features." *Expert Systems with Applications*, vol. 36, no. 2, 2009, pp. 3302-3308.
- Breiman, Leo, Jerome Friedman, Richard Olshen, and Charles Stone. *Classification and Regression Trees*. Wadsworth, 1984.
- Cortes, Corinna, and Vladimir Vapnik. "Support-Vector Networks." *Machine Learning*, vol. 20, no. 3, 1995, pp. 273-297.
- Litterman, Robert, and Jose Scheinkman. "Common Factors Affecting Bond Returns." *The Journal of Fixed Income*, vol. 1, no. 1, 1991, pp. 54-61.
- Pearson, Karl. "On Lines and Planes of Closest Fit to Systems of Points in Space." *Philosophical Magazine*, vol. 2, no. 11, 1901, pp. 559-572.
- Syed Nor, Shafinar, Sharifah Heryati Ismail, and Bee Wah Yap. "Personal Bankruptcy Prediction Using Decision Tree Model." *Journal of Economics, Finance and Administrative Science*, vol. 24, no. 47, 2019, pp. 157-170.

Issue 3: Applying Ensemble Learning - Bagging, Boosting, and Stacking

Technical Section

3.1 Purpose of ensemble learning

- An ensemble combines forecasts from several fitted learners into one prediction. When component errors are not perfectly correlated, averaging can reduce variance. Different learners can also capture different structures, such as linear boundaries, nonlinear interactions, and threshold effects.
- This project predicts whether the next daily close is higher using lagged returns, momentum, realised volatility, moving-average distance, RSI, and volume change. The first 75% of observations are used for fitting and time-series validation; the final 25% remain held out until the end.

3.2 Main ensemble approaches

- Bagging fits many versions of a base learner on resampled training data and averages their probabilities. Random forests are a homogeneous bagging method: bootstrap samples and random feature subsets make the trees less correlated, so their average is more stable than a single deep tree.
- Boosting builds learners sequentially. Each new learner focuses on the residual error of the current combined model. Important controls are the number of stages, learning rate, tree depth, and minimum leaf size.

$$FM(x) = F0(x) + \eta \sum_{m=1}^M hm(x)$$

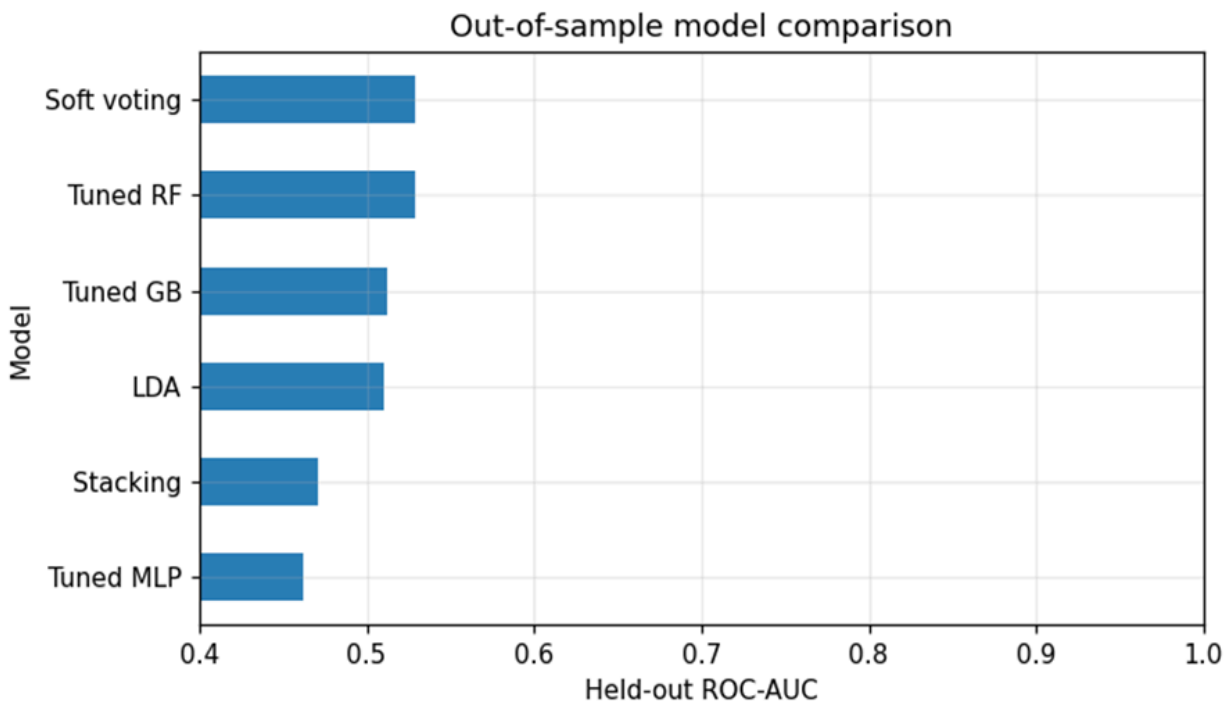
- Stacking trains a second-level model on out-of-fold base predictions, preventing the meta-model from learning in-sample noise. In this report the stacker uses a regularised logistic layer.

$$\hat{y}_i = g(z_{i1}, z_{i2}, \dots, z_{ik})$$

3.3 Models and hyperparameters used

- The base layer contains LDA, a tuned neural network, a tuned random forest, and gradient boosting. Settings were selected inside the training period using four expanding-window folds and ROC-AUC. Soft voting averages component probabilities with equal weights; stacking uses five internal folds for its second-level training.

Model	Key setting	Role
LDA	Linear discriminant	Low-variance reference
Tuned MLP	16 hidden units; $\alpha=0.01$	Flexible nonlinear reference
Random forest	100 trees; leaf size 8	Bagging
Gradient boosting	50 stages; depth 2	Boosting
Stacking	Regularised logistic meta-model	Out-of-fold combination

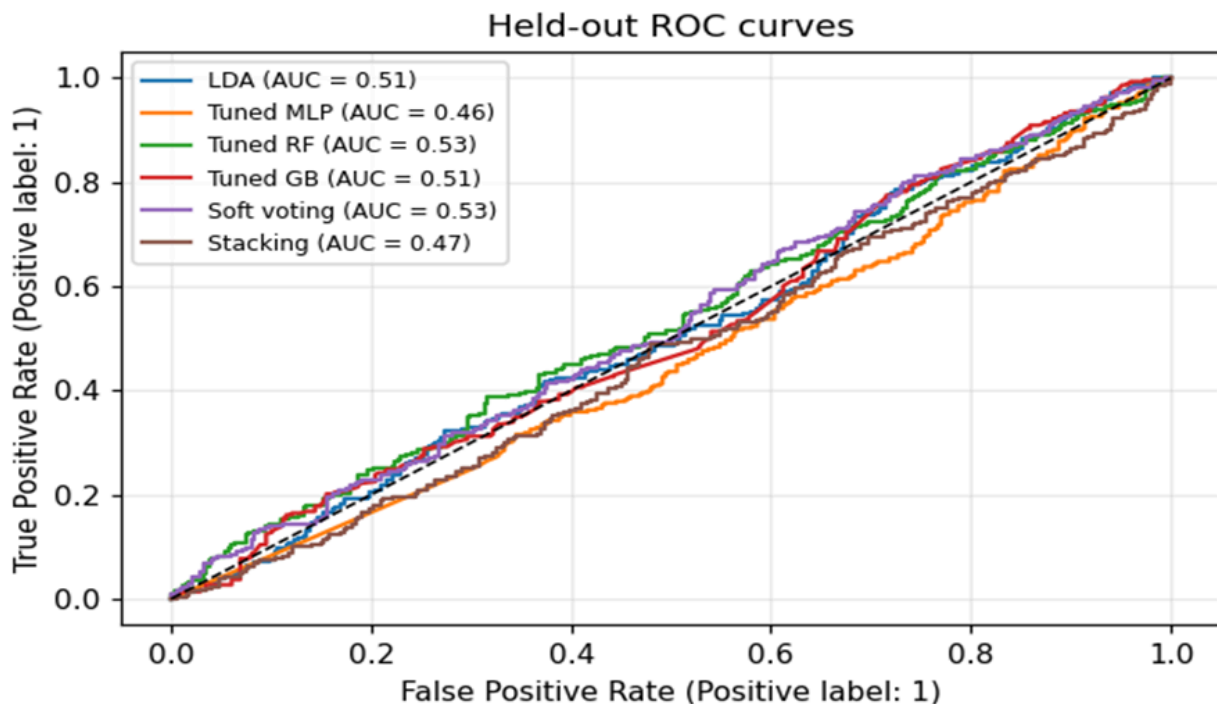


3.4 Quantitative results

- The following results are from the untouched chronological test period. Accuracy is the fraction of correct classifications; precision, recall, and F1 describe the positive class; ROC-AUC measures ranking quality across probability thresholds.

Model	Accuracy	ROC-AUC	Precision	Recall	F1
LDA	0.4954	0.5105	0.4686	0.7013	0.5618
Tuned MLP	0.4505	0.462	0.439	0.6879	0.5359
Tuned RF	0.5046	0.5293	0.4703	0.5839	0.521
Tuned GB	0.5046	0.5118	0.4776	0.7886	0.5949
Soft voting	0.5093	0.5293	0.478	0.6913	0.5652
Stacking	0.4644	0.471	0.4542	0.7987	0.5791

- The soft-voting ensemble produced the best accuracy, 50.93%, and tied the random forest for the strongest ROC-AUC at 0.5293. Stacking had the highest recall, 79.87%, but lower accuracy and ROC-AUC. The results therefore provide modest evidence for combining forecasts, not evidence of a strong daily forecasting edge.



Non-Technical Section

- The ensemble is like asking several analysts for an opinion before making one decision. Combining forecasts can make the decision less sensitive to one analyst's mistake, but the strongest combined result correctly classified only about 51 out of every 100 later observations. That is close to an even split and is not enough to justify live capital.
- Recommended course of action: use soft voting as the starting candidate for a paper-trading pilot; require a confidence threshold; use small volatility-adjusted positions; include spreads, commissions, slippage, turnover, and liquidity limits; and compare against a simple benchmark and cash. Disable the strategy if live calibration falls materially outside the historical range.
- Factors affecting outcomes include transaction costs, market impact, liquidity, volatility regimes, class imbalance, data leakage, missing observations, and changing market structure. Directional accuracy is not the same as profitable trading performance.

References

Breiman, Leo. Random Forests. *Machine Learning*, vol. 45, 2001, pp. 5-32. Friedman, Jerome H. Greedy Function Approximation: A Gradient Boosting Machine. *The Annals of Statistics*, vol. 29, 2001, pp. 1189-1232. Hastie, Trevor, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. 2nd ed., Springer, 2009. Wolpert, David H. Stacked Generalization. *Neural Networks*, vol. 5, 1992, pp. 241-259.